

FedSepsis-KG: GAT Development and Implementation Plan

Scope: PhysioNet/CinC 2019 Phase 4 graph experiments, while preserving the frozen Phase 3 GRU-D and Transformer baselines and the future eICU-CRD transfer design.

Core decision: do not modify the current GAT-0 or the current 35-feature baseline now. First finish and evaluate Phase 3. Then audit GAT-0, implement GAT-1 as a spatial clinical GAT applied at every hour followed by a causal GRU, and only later implement explicit temporal graph edges and Medical-KG integration.

1. What already exists

- Phase 3 data contract: values, masks, deltas, static_features, valid_mask.
- 35-feature baseline schema, including ICULOS, with 105 concatenated value/mask/delta channels.
- Training-only preprocessing artifacts, patient-level splits, static features, and timestep labels.
- GRU-D baseline intended as the missingness-aware temporal baseline.
- Temporal Transformer baseline intended as the causal temporal-attention baseline.
- A provisional static correlation-based GAT result. Preserve it; do not overwrite it.
- A correlation graph idea connecting clinical variables. This is a static feature graph, not yet a temporal graph or Medical Knowledge Graph.

2. What must not be touched now

- Do not change the 35-feature Phase 3 schema.
- Do not remove or separately encode ICULOS in the current baseline.
- Do not change the current GRU-D architecture or checkpoint.
- Do not change the current Transformer architecture while completing its baseline.
- Do not rebuild the graph using validation or test patients.
- Do not introduce PyTorch Geometric unless the existing project already depends on it.
- Do not build a B×T×F temporal graph as the first new experiment.
- Do not add a Medical KG before the correlation-GAT experiments are stable.
- Do not compare models with different splits, preprocessing artifacts, labels, or loss definitions.

3. Terminology and experiment progression

Experiment	Graph	Temporal module	ICULOS
GRU-D baseline	None	GRU-D	35-feature baseline
Transformer baseline	None	Causal Transformer	35-feature baseline
GAT-0	Static feature-correlation graph	Current/minimal readout	35-feature baseline
GAT-1	Static clinical graph at each hour	Causal GRU	35-feature baseline
GAT-1T	Static clinical graph at each hour	Causal Transformer	35-feature baseline
GAT-2	Clinical + past-to-current temporal edges	Temporal GAT	35-feature baseline initially
KG-GAT	Medical/domain graph	Temporal encoder	Controlled later variant

4. GAT-0: freeze and document

GAT-0 is the existing static correlation-based feature GAT. Its purpose is to answer whether relationships among clinical variables help. It is not temporal GAT and not Medical-KG GAT. Preserve its checkpoint, metrics, graph construction, and configuration.

GAT-0

```

nodes = clinical_variables
edges = training-only correlation graph
input at time t = [value, observation_mask, delta]
readout = current implementation
label = timestep-level SepsisLabel

```

Before accepting GAT-0 as a baseline, record: feature order, graph source, correlation method, threshold, self-loops, normalization, checkpoint date, split hash, preprocessing-config hash, AUROC, AUPRC, and official challenge utility score if available.

5. GAT-1: recommended next implementation

GAT-1 keeps the 35-feature input and applies the same frozen clinical correlation graph independently at every real timestep. It then pools feature nodes with learned attention and sends the resulting hourly graph representations through a causal GRU. This isolates the effect of adding temporal modeling to the static feature graph.

```

values, masks, deltas: [B, T, F], F=35
node_x[t] = stack([values[:,t,:], masks[:,t,:], deltas[:,t,:]], dim=-1)
node_x[t]: [B, F, 3]
node_h[t] = node_projection(node_x[t])
node_h[t]: [B, F, H]
node_h[t] = GAT(node_h[t], A_clinical)
score[t,i] = w^T tanh(W node_h[t,i])
alpha[t] = softmax(score[t], dim=feature_nodes)
graph_t = sum_i alpha[t,i] node_h[t,i]
graph_sequence: [B, T, H]
gru_out = causal_GRU(graph_sequence)

```

6. Mathematical specification

For clinical feature i at time t : $x_{t,i} = [v_{t,i}, m_{t,i}, \delta_{t,i}] \in \mathbb{R}^3$. The node projection is $h_{t,i}^{(0)} = W_{\text{node}} x_{t,i} + b_{\text{node}}$.

For edge $j \rightarrow i$ in the frozen clinical graph: $e_{t,ij} = \text{LeakyReLU}(a^T [W h_{t,i} \parallel W h_{t,j}])$; $\alpha_{t,ij} = \exp(e_{t,ij}) / \sum_{k \in N(i)} \exp(e_{t,ik})$; $h'_{t,i} = \sigma(\sum_{j \in N(i)} \alpha_{t,ij} W h_{t,j})$.

Learned node readout: $s_{t,i} = w_r^T \tanh(W h'_{t,i})$, $\alpha_{t,i} = \text{softmax}_i(s_{t,i})$, $g_t = \sum_i \alpha_{t,i} h'_{t,i}$. Causal GRU: $h_t = \text{GRU}(g_t, h_{t-1})$. Use h at the last valid timestep, not the last padded position.

7. Training-only graph construction

Build the graph using training patients only, then freeze it for validation and test. Calculate feature correlations only on the training partition. Choose the method and threshold before inspecting validation/test results. Include self-loops.

Save the exact feature order, correlation method, threshold, adjacency/edge index, and graph hash.

```

train_only = load_training_clinical_values(...)
corr = compute_pairwise_correlation(train_only)
adj = (abs(corr) >= threshold).astype(np.float32)
adj = add_self_loops(adj)
np.save("artifacts/gat0_adjacency.npy", adj)
# validation and test use exactly this saved adj

```

Do not calculate correlations from all files, validation data, or test data. If pairwise correlation uses imputed data, document the imputation and run a sensitivity analysis because imputation can affect graph edges.

8. Dense PyTorch GAT layer

```

class DenseGATLayer(nn.Module):
    def __init__(self, in_dim, out_dim, dropout=0.1):
        super().__init__()
        self.lin = nn.Linear(in_dim, out_dim, bias=False)
        self.attn = nn.Parameter(torch.empty(2 * out_dim))
        self.leaky = nn.LeakyReLU(0.2)
        self.drop = nn.Dropout(dropout)
        nn.init.xavier_uniform_(self.lin.weight)
        nn.init.xavier_uniform_(self.attn.unsqueeze(0))

    def forward(self, x, adj):
        # x: [B, F, H]
        z = self.lin(x)
        left = (z * self.attn[:z.size(-1)]).sum(-1)
        right = (z * self.attn[z.size(-1):]).sum(-1)
        e = self.leaky(left.unsqueeze(-1) + right.unsqueeze(-2))
        e = e.masked_fill(adj.unsqueeze(0) == 0, float("-inf"))
        alpha = torch.softmax(e, dim=-1)

```

```

alpha = self.drop(alpha)
out = torch.matmul(alpha, z)
return torch.elu(out), alpha

```

For $F=35$, a dense adjacency implementation is practical and avoids a new dependency. Keep adjacency on the same device as the node tensor.

9. GAT-1 model skeleton

```

class GATTemporal(nn.Module):
    def __init__(self, input_size=35, static_size=5, node_dim=64,
                  gru_hidden=64, dropout=0.3):
        super().__init__()
        self.node_projection = nn.Sequential(
            nn.Linear(3, node_dim), nn.GELU(), nn.LayerNorm(node_dim)
        )
        self.gat = DenseGATLayer(node_dim, node_dim, dropout)
        self.node_score = nn.Linear(node_dim, 1)
        self.gru = nn.GRU(
            input_size=node_dim, hidden_size=gru_hidden,
            batch_first=True
        )
        self.static_projection = nn.Sequential(
            nn.Linear(static_size, gru_hidden), nn.GELU(),
            nn.LayerNorm(gru_hidden)
        )
        self.classifier = nn.Linear(2 * gru_hidden, 1)

    def forward(self, values, masks, deltas, static, valid_mask, adj):
        B, T, F = values.shape
        graph_seq = []
        for t in range(T):
            node_x = torch.stack(
                [values[:, t, :], masks[:, t, :], deltas[:, t, :]],
                dim=-1
            ) # [B,F,3]
            node_h = self.node_projection(node_x)
            node_h, _ = self.gat(node_h, adj)
            score = self.node_score(torch.tanh(node_h)).squeeze(-1)
            alpha = torch.softmax(score, dim=-1)
            graph_t = (alpha.unsqueeze(-1) * node_h).sum(dim=-1)
            graph_t = graph_t.masked_fill(
                ~valid_mask[:, t].unsqueeze(-1), 0.0
            )
            graph_seq.append(graph_t)

        graph_seq = torch.stack(graph_seq, dim=1)
        lengths = valid_mask.sum(dim=1).long().clamp_min(1)
        packed = nn.utils.rnn.pack_padded_sequence(
            graph_seq, lengths.cpu(), batch_first=True, enforce_sorted=False
        )
        packed_out, _ = self.gru(packed)
        gru_out, _ = nn.utils.rnn.pad_packed_sequence(
            packed_out, batch_first=True, total_length=T
        )
        idx = (lengths - 1).to(values.device)
        temporal = gru_out[torch.arange(B, device=values.device), idx]
        static_h = self.static_projection(static)
        return self.classifier(torch.cat([temporal, static_h], dim=-1)).squeeze(-1)

```

The skeleton returns one patient-level logit. If labels are hourly, use a timestep classifier on each GRU output $[B,T,H]$ and mask BCE. Do not mix patient-level logits with hourly labels.

10. Padding and mask rules

- Feature observation mask: $[B,T,F]$, part of node input.
- `valid_mask`: $[B,T]$, True for real timesteps and False for padding; never a graph feature.
- Padded graph outputs must be zeroed or excluded.
- Use packed sequences or a valid-mask-aware temporal readout.
- Never use `graph_seq[:, -1, :]` unless every sequence has the same length.
- Never allow future information into a causal prediction.

11. GAT-2: defer until GAT-1 is stable

GAT-2 creates feature-time nodes (i,t) , retains within-time clinical edges, and adds past-to-current edges $(i,t-k) \rightarrow (i,t)$. Do not add future edges $(i,t+1) \rightarrow (i,t)$. This is a separate experiment, not a modification of GAT-1.

12. Medical KG integration: later

A correlation graph is data-derived and statistical. A Medical KG is knowledge-derived and semantic. Do not call GAT-0 a Medical-KG model. Later compare no graph, correlation graph, and curated KG using identical preprocessing and evaluation.

13. Verification checklist

- GAT-0 checkpoint and metrics are backed up and documented.
- GAT-1 uses the same 35-feature preprocessing and split artifacts as Phase 3.
- $F=35$; node input is $[B,F,3]$.
- Correlation graph is built from training patients only and frozen.
- Adjacency has self-loops and no accidental future/time edges in GAT-1.
- GAT runs at each real timestep.
- Feature-node attention pooling is used or explicitly documented.
- `valid_mask` is separate from the clinical observation mask.
- Padded graph outputs do not affect the GRU.
- GRU readout uses last valid timestep or masked pooling.
- Label shape matches output shape and loss level.
- All logits and losses are finite.
- Graph hash, feature order, preprocessing hash, seed, threshold, and metrics are saved.
- Official PhysioNet utility score is reported in addition to AUROC/AUPRC.

14. Final implementation order

1. Finish Transformer 35-feature baseline and official evaluation.
2. Freeze Phase 3 artifacts and metrics.
3. Audit current GAT-0 without changing it.
4. Build and freeze the training-only correlation graph.
5. Implement GAT-1: per-hour GAT $\rightarrow$ learned node pooling $\rightarrow$ causal GRU.
6. Run GAT-1 with identical splits, preprocessing, and evaluation.
7. Only after GAT-1 works, compare GAT-1T using a causal Transformer.
8. Then implement GAT-2 explicit lag edges.
9. Then introduce and evaluate the Medical KG.
10. Only afterward attempt Transformer–GAT fusion and federated/personalized learning.

15. Bottom line

The next implementation is GAT-1, not a giant temporal graph and not a Medical KG. Keep GAT-0 untouched, keep the 35-feature baseline, derive the graph from training patients only, apply the same spatial graph at every hour, use learned feature-node pooling, then use a causal GRU with padding-aware readout. This gives a clean progression from static clinical relationships to temporal clinical dynamics.

Research note: temporal graph models explicitly represent evolving neighborhoods or time-dependent node states, while a static correlation graph only represents a fixed relation among variables. The staged design separates these claims and prevents attributing temporal gains to an unverified graph change. [web:135][web:141]